Open TechEd: Revival of Legacy Teaching Instruments for Modern Education

A pilot on the Thermo Fisher ND1000 (September 2026)

Jord van Leeuwen,[1] Vincent de Boer,[2] and Vittorio Saggiomo[1]

1 Department of BioNanoTechnology

2 Department of Human and Animal Physiology

Abstract

Legacy scientific instruments are often discarded not because their hardware has failed, but because the proprietary software that controls them is no longer supported on modern operating systems. This creates unnecessary e-waste and is especially problematic in teaching environments, where universities must replace perfectly functional instruments. In this project we show that such instruments can be brought back to life through reverse engineering. Using the Thermo Fisher NanoDrop 1000 microvolume spectrophotometer as a pilot, we reverse-engineered its undocumented USB communication protocol by sniffing the traffic of the proprietary software with Wireshark. We identified the internal Ocean Optics USB2000 spectrometer as the optical core and decoded the commands controlling the xenon lamp, the solenoid clamp, and the data transfer, including wavelength calibration and pixel de-interleaving. We then wrote open-source Python software to control the instrument and integrated it into PyLabRobot, an open-source SDK for laboratory automation. The instrument was validated by measuring a xenon lamp spectrum that closely matches the literature and by detecting paracetamol in water. Our work shows that a legacy instrument can be revived into a fully usable, open piece of laboratory equipment, available to anyone, anywhere, as part of an automated laboratory.

Contents

Introduction.....2

Is the instrument you bought really yours?.....2

The Thermo Fisher NanoDrop 1000 Spectrophotometer.....3

Reverse Engineering the Black Box.....5

Materials and Methods.....6

Hardware.....6

Software.....6

Pipeline.....6

Results and Discussion.....7

Installation on later Windows versions.....7

The data.....9

Conclusions.....11

Links to the project:.....12

Sources.....13

Introduction

Is the instrument you bought really yours?

The question may seem strange when it comes to physical hardware: the instrument sits in your laboratory, you can see it, you can touch it, clearly, it is yours. But, leaving aside some peculiar exceptions, a Nintendo Switch tied to an account, or the bank card in your wallet (which technically belongs to and can be controlled by the bank) are more well-known examples where the hardware can be controlled by another party than yourself and is legally owned by them.

In essence you are just “permanently renting” the equipment. That point settled, let us move to the second one: can you freely use *your* instrument?

And here things get more complicated. To control any modern piece of hardware you need software to run it. Here we define “modern” as needing software, as for example you can still take apart and make an old Fiat 500 work just by using screwdrivers and wrenches, while you need a computer and software to fix any modern car. Usually, that software is developed and owned by the same company that built the hardware. In many, many cases, this software is **not** yours. You are allowed to use it, but you do not own it. This illustrates the fact that in the modern world, most hardware and software is proprietary, and thus you technically do not own the product, you have it on loan and may use it, provided you have the license.

And with this non-ownership come many problems:

Subscriptions. It is clear why companies nowadays prefer to sell subscriptions (Adobe, Microsoft) rather than the software itself. First off, this keeps the software and its updates under their rights, disincentivizing open-source forks. And secondly, it is a lucrative business, as renting out software over time procures more money than a one-time lifetime update guaranteed package. It is not hard to guess where this is heading: sooner or later, you may be paying a monthly fee just to use the instruments in your own laboratory.

Digital sovereignty. If the software must be connected to the internet for verification, the company has the ability to lock your instrument remotely (as it has already happened with some gaming consoles or cars).[1,2] And if your data is stored "in the cloud," the company can use that data, or lock you out of it.[3] This means that the company has full control over your access to the product, regardless of whether you paid for it.

Interoperability. If the software is proprietary, which it often is, the instrument cannot (easily) be controlled by any other software. This is a growing problem with, for instance, the rise of self-driving labs, where multiple instruments are connected together and controlled autonomously by algorithms. If an instrument is locked to a single piece of software, this automation becomes much more difficult.

Legacy instruments and e-waste. If a developer declares an instrument "legacy," they will no longer develop software for it. This pushes laboratories to buy new instruments even though the old hardware still works flawlessly. For example, if an instrument only runs on Windows XP, some laboratories must keep a dedicated Windows XP computer just to use it.[4] This could pose a serious IT and security problem, since those machines are never updated and vulnerabilities are well-known. Moreover, this way of working creates a great deal of unnecessary e-waste. **This is especially problematic in a teaching**

environment, where a university buys many instruments for teaching and, after a few years, has to throw them away simply because the software is no longer supported.

As discussed above, there is strong pressure for Open Science from many directions. Open-Source Software is already well advanced, and Open-Source Hardware is slowly growing. But the connection between these two, software and hardware, is fundamental if we want to do open science and to own both our instruments and our data.

In this project, we pilot the possibility of picking up a legacy instrument and bringing it back to life by reverse-engineering it and writing Python software to control it. As this project was funded by Open Science Education, we focused on the last point (legacy instruments and e-waste), although this work also addresses all the other points.

Obtaining a proper list of legacy hardware, instruments that still work but can no longer be used on the Wageningen University & Research (WUR) network because of outdated software, from the Practicum Facilitaire Ondersteuning (PFO) proved impossible. However, from a conversation with them we learned that the Thermo Fisher NanoDrop 1000 spectrophotometer is an instrument they used to use, but due to outdated software that is no longer possible. As we also had a couple of these instruments lying around, we decided to run the pilot on this instrument.

The Thermo Fisher NanoDrop 1000 Spectrophotometer

The NanoDrop 1000 (ND-1000) is a full-spectrum UV-Vis microvolume spectrophotometer. It was introduced in 2001 by NanoDrop Technologies, a company later acquired by Thermo Fisher Scientific. Before the NanoDrop, measuring nucleic acids required diluting concentrated samples, pipetting them into cuvettes, and running laborious multi-step protocols. The NanoDrop 1000 did away with all of that: a sample of just 1–2 µL is pipetted directly onto a fiber-optic measurement surface, where it is held in place by surface tension alone. No cuvettes, no capillaries, no dilution, and a full spectrum in about ten seconds.

The instrument is a full-spectrum spectrophotometer covering 220–750 nm. It uses a xenon flash lamp and a 2048-element linear silicon CCD array detector with no moving parts, giving it high reliability and a wavelength accuracy of ±1 nm. Two path lengths (2 mm and 0.2 mm) are used automatically within each measurement cycle, which gives the instrument an enormous dynamic range: it can measure dsDNA from a detection limit of about 2 ng/µL up to roughly 3,700 ng/µL without any dilution.

In practice, the NanoDrop 1000 is a workhorse for routine quality checks in molecular biology. Its most common applications are quantifying nucleic acids (DNA and RNA) and assessing their purity using absorbance ratios such as A260/A280, measuring purified protein concentration (A280), running colorimetric protein assays (Bradford, BCA, Lowry), measuring fluorescent dye labelling density, and general UV-Vis spectrophotometry. Because it needs only a microliter of sample and returns an answer in seconds, it is ideal for the fast, everyday measurements needed in both research and teaching laboratories.

When new, the NanoDrop 1000 was a significant investment, it originally sold for roughly €15,000-€20,000 (an approximate figure, as official pricing is no longer published for this discontinued model). Today, second-hand units are widely available for around €3,000-€3,500, with refurbished units (often bundled with a laptop and software) ranging up to about €6,000.

The main problem is that the NanoDrop 1000 is controlled from a Windows PC through a USB connection, using the dedicated "NanoDrop 1000 Operating Software" (version 3.8.1, last released in June 2011). The software is free to download from Thermo Fisher, but it requires registration. Critically, the manufacturer only supports the software on older versions of Windows: officially, it runs on Windows 7 Professional (32- and 64-bit), Windows Vista (32-bit), and Windows XP (32-bit). There is no support for modern versions such as Windows 10 or Windows 11, precisely the kind of "legacy" situation described in the introduction. The hardware still works, but the software that controls it is effectively stranded on outdated operating systems.

And this is exactly why we chose the NanoDrop 1000 for this project. The instrument is a perfect candidate for reverse engineering: it is well documented, communicates with the computer over a simple USB connection, and its hardware is still fully functional. Our goal is to reverse-engineer the communication protocol and write our own open-source Python software to control the instrument, so that it can be used on any modern machine, Windows, macOS, or Linux, without depending on the vendor's obsolete software. In doing so, we aim to turn a "legacy" instrument that was headed for the scrap heap into a fully usable, open piece of laboratory equipment.

To put the problem in financial terms, consider a simple imaginary scenario. Suppose a university bought ten NanoDrop 1000 instruments three years after the model launched in 2001, so in 2004, at roughly €20,000 each. That was a €200,000 investment in 2004. Equipment like this is normally written off over ten years, so the instruments were fully depreciated by 2014. The hardware runs much longer than that. In practice a spectrophotometer of this kind lasts twenty years or more. This is where the real value sits: once the instruments are paid off, they keep measuring at no extra cost for another decade. The end of software support takes that value away. The final software update for the NanoDrop 1000 was released in June 2011, and from the release of Windows 8 in October 2012 the device could no longer reliably communicate with modern operating systems. In other words, only seven to eight years after purchase, an investment of €200,000 (in 2004) could no longer be used, not because the hardware had failed, but solely because the vendor had stopped supporting the software.

To connect our instrument to the wider world of automated laboratories, we build on PyLabRobot. PyLabRobot is an open-source, pure Python Software Development Kit (SDK) for automated and autonomous laboratories. It is hardware- and operating-system-agnostic, meaning it provides a single, universal Python interface to control many different laboratory robots and accessories, regardless of the computer or operating system they run on. Instead of each instrument being locked to its own proprietary software, PyLabRobot lets researchers control and orchestrate a whole suite of instruments from one place, which is exactly what is needed for the self-driving laboratories described in the introduction. By integrating the NanoDrop 1000 into the PyLabRobot ecosystem, we aim to make our reverse-engineered control software available to everyone, so that anyone, anywhere, can plug in a NanoDrop and use it as part of their own automated lab.

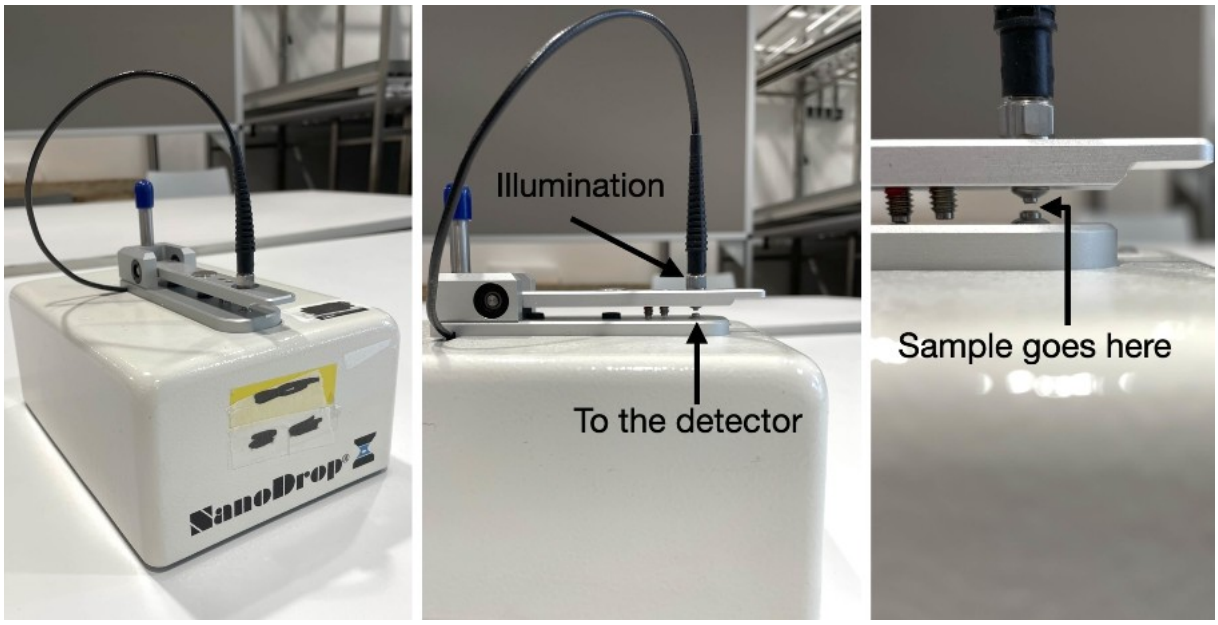


Figure 1: The Thermo Fisher NanoDrop 1000, and the highlight of its detection zone.

Reverse Engineering the Black Box

A standard way to reverse engineer the communication protocol of an instrument is by observing the external USB traffic (black-box reverse engineering). It is not a simple task, however, as the proprietary software usually uses custom names for protocols and probably uses different languages than Python. One aspect that we can take full advantage of, however, is the fact that serial traffic (what the USB device and your computer use to communicate), is completely free. We are free to send arbitrary commands over serial to anything we plug into our computer, and if those "arbitrarily chosen" bits happen to activate a command on our USB device, be it proprietary or not, it is fine to repeat this however we want. This is the basis for reverse engineering,[5] where we identify commands necessary to operate the USB device, write a program to do it for us, and ultimately create a program similar to the proprietary software, albeit in Python and open-source.

The internal optical component is a modified Ocean Optics USB2000 spectrometer, as discovered earlier by a teardown.[6] This removes a massive wall for open-source programming, as the Ocean Optics component has regular documentation. The only remaining unknown is how Thermo Fisher integrated it into their protocol and if they use any proprietary components, which we lack documentation of. By analysing USB packet captures via Wireshark (a software that lets you see the USB communication) and cross-referencing OEM data sheets, it is possible to crack the undocumented communication protocol and write a custom Python script emulating the necessary commands.

Once the necessary commands are reverse engineered, the remaining thing to do would be to create protocols. This is the automatic application of the commands, such as: clamp down the illumination block (Figure 1), turn the light on, measure, read data, together with the follow-up of the data. This includes the cleaning, and the algorithms to transform the data from a spectrum to, for instance, a DNA or protein concentration.

Materials and Methods

Hardware

When identifying proprietary protocols, it is necessary to operate the device using regular means, therefore for this project it was not only needed to use the NanoDrop 1000, but also a working computer on Windows 7 was used. To identify bits sent over serial it was first attempted to use the Beagle USB 12 protocol analyser.[7] This is a non-intrusive hardware analyser, which captures packet data sent over USB. This device is perfect, as it allows full reading of all packets sent via a second computer which is not running on Windows 7, while the test device can run the Thermo Fisher program on native Windows 7. However, the sheer amount of packets was immense, therefore it was opted to switch over to Wireshark software later.[8]

Software

As mentioned above, the original software was required. Luckily this software is free to download from the Thermo Fisher website, and since we already own an original key, the software was operable. With some extra steps it was also possible to run the original software on later versions of Windows, however this still requires an original key, whereas second-hand devices, or some with simply “lost to time” keys will not be operable this way.

The Beagle has its own software downloadable for free, it is required to operate the Beagle to obtain the USB data information. The alternative, Wireshark, is also available for free, and can be used natively on Windows without any additional hardware requirements.

Most devices need a driver; this software bridges the communication between the device and the operating system. The NanoDrop has its own drivers which come installed with the software, however, to use the device over Python it is necessary to install libusb-win32, which is a driver that the PyUSB library recognizes. This can be achieved by using the free-to-download program Zadig, which easily swaps the driver. Not to worry about losing the original functionality, as the driver can be swapped back to the original if required via the native Windows driver installations.

Pipeline

The pipeline consists of using the original software, such as the startup sequence, sniffing it using the software, and recording the serial traffic (Figure 2). This traffic is recorded as a series of hexadecimal strings, and it is recommended to use an artificial intelligence (we used Google Gemini and Claude Code) to parse through the vast array of hexadecimal bits to identify patterns. From these patterns, common bytes can be extracted, such as the bytes meaning “lamp on.” Another way to find these bit commands is via the factory manual, however in the case of reverse engineering, this is not always available (such is the case for the non-Ocean Optics parts). Once these bytes are known, it is possible to send them via Python script over serial, enabling the same command to be used without needing the proprietary software. This creates a new program which sends identical commands to the device as the proprietary software.

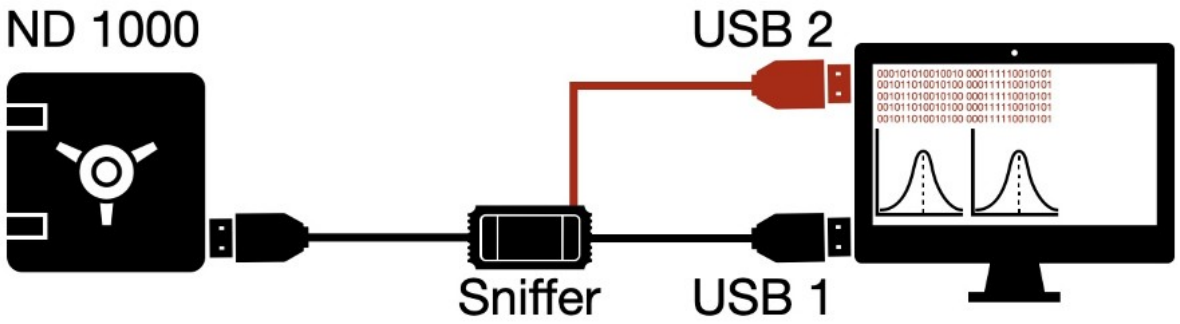


Figure 2: Standard scheme of a USB "sniffer": while the standard program is used to control the instrument via the USB 1, at the same time, the USB 2 is used to record the USB traffic and will be used later to decode which command is used to do what on the instrument. USB 2 may be connected to a different computer than the one the software is running on.

Results and Discussion

Installation on later Windows versions

Because this device relies on a legacy USB 1.1 microcontroller, Windows 10/11 requires specific workarounds. This means that it is fully possible to run the original software on newer Windows versions. Note that this still requires a valid key and it will require the Binary Device Object Store (BOS) descriptor fix as well as unpacking the software and running the install programs in a specific order:[9]

1. Extract the downloaded setup package (e.g., ND1000_3.8.1.exe)
2. Open the extracted folder and run the component installers in this exact order:
3. ND1000/LVRuntime7/lvruntimeeng71.msi
4. ND1000/NI-VISA 4.4.1/setup.exe
5. ND1000/NanoDrop 1000 V3.8.1.msi
6. 64bitdriver/NanoDrop 1000 64bit Driver.msi

Next the BOS descriptor needs to be circumvented. The NanoDrop 1000 utilizes an older USB 1.1 Cypress microcontroller (as long as they did not add an additional controller other than the Ocean Optics one).[10] Modern operating systems (Windows 8 and newer) query newly connected USB devices for a BOS descriptor to determine if they support SuperSpeed or advanced power management.[11] Because the BOS descriptor was not introduced until the USB 3.0 and USB 2.0 Low Power Mode (LPM) specifications, the legacy NanoDrop microcontroller does not understand this request, causing the hardware handshake to stall and the device to fail enumeration. The included .reg file fixes this by setting the SkipBOSDescriptorQuery flag in the Windows Registry, explicitly instructing the operating system to skip this modern check for the NanoDrop's specific Vendor and Product ID (PID and VID).

This VID and PID is mandatory for all USB devices and can easily be found in the Windows Device manager. It might be possible that the PID and/or VID are somehow changed in another device, although this should not be the case, which also showed in testing where two different devices had the same VID and PID. A step by step tutorial is given below to manually change the VID and PID in the .reg:

1. Open the Windows **Device Manager**.
2. Locate the connected device (often found under *Universal Serial Bus controllers*, *Unknown Devices*, or *libusb-win32 devices*).

3. Right-click the device and select **Properties**.
4. Navigate to the **Details** tab.
5. Click the "Property" dropdown menu and select **Hardware Ids**.
6. You will see a string of text resembling USB\VID_2457&PID_1002. The four alphanumeric characters after VID_ represent the Vendor ID, and the four characters after PID_ represent the Product ID.

To allow the PyUSB library to operate the device, the program Zadig was necessary to change the driver from the proprietary OEM to the standard libusb-win32 driver. This means the only real dependency of the backend is PyUSB for OS-agnostic communication. Luckily it is possible to switch back drivers seamlessly. The steps to install the open source driver and reinstall the proprietary driver can be found in the NanoDropInstallInstructions.txt in the GitHub associated with this report.[12]

The USB sniffing and hardware manual left us with some insights of which we concluded a few things for the software to work:

- **Mailbox Mapping:** Discovered that Endpoint 0x02 handles commands, Endpoint 0x87 handles text/status communication, and Endpoint 0x82 delivers the heavy raw image data. These mailboxes are what Thermo Fisher uses, and were not changed from what is documented on the USB2000. This was evident from the Wireshark readings, but could also be cross-referenced with the Ocean Optics manual.
- **Wavelength Calibration:** Successfully extracted the factory-set 3rd-order polynomial coefficients $C_0 C_3$ directly from the hardware's EEPROM to construct a scientifically accurate nanometer X-axis.[10,13] This polynomial is device specific and thus needs to be extracted as detailed in the USB2000 data sheet. Another 7th order correction polynomial is available for non-linearity correction but in early, quick and dirty testing it was decided to only use the first 3rd order polynomial, which is necessary for wavelength calibration. In the later updates on PyLabRobot all coefficients are downloaded.
- **Data De-interleaving & Pixel Decoding:** The internal spectrometer utilizes a 12-bit Analog-to-Digital Converter (ADC) to read the light intensity of its 2048 pixels. Because a 12-bit value cannot fit into a standard 8-bit byte, the data for each pixel is split across two bytes: a Least Significant Byte (LSB) and a Most Significant Byte (MSB). Instead of transmitting these bytes sequentially, the USB controller optimizes bandwidth by sending them in interleaved 64-byte blocks. Packet 0 contains the LSBs for the first 64 pixels, and Packet 1 contains the corresponding MSBs for those same pixels. Reconstructing the spectrum required a custom Python algorithm to stitch these blocks back together, calculating the true intensity of each pixel (from 0 to 4095 counts) using the standard bitwise shift operation: Intensity = (MSB << 8) | LSB, which means the MSBs get shifted 8 bits (or 1 byte) to the left and the LSBs get pasted after it. This is also documented on the USB2000 data sheet, but reconstructing it required significant insight and left us stumped during early attempts…
- **Xenon Lamp (0x03):** The Ocean Optics USB2000 manual documents Command 0x03 as the "Set Strobe Enable Status" command, which toggles the Lamp Enable line (J2 Pin 4) to HIGH/On (0xFF) or LOW/Off (0x00). Thermo Fisher utilized this exact strobe circuit to fire their custom

Xenon flash module, which was identified using Wireshark even before cross-referencing it in the Ocean Optics manual.

- **Solenoid Clamp (0x0F):** Unlike the lamp, the command to actuate the magnetic pedestal clamp (0x0F) does not exist in the official Ocean Optics USB2000 command set. Because this solenoid is proprietary Thermo Fisher hardware added around the optical bench, its control hex code had to be intercepted and identified purely by sniffing the USB traffic of the proprietary Windows software using Wireshark. Much like the lamp, the clamp also has a standard HIGH/On (0xFF) or Low/Off (0x00) configuration.

The data

To check if the instrument worked properly on Python, we checked the illumination system, meaning that we recorded the spectra of the Xenon lamp and compared it to one found in literature.

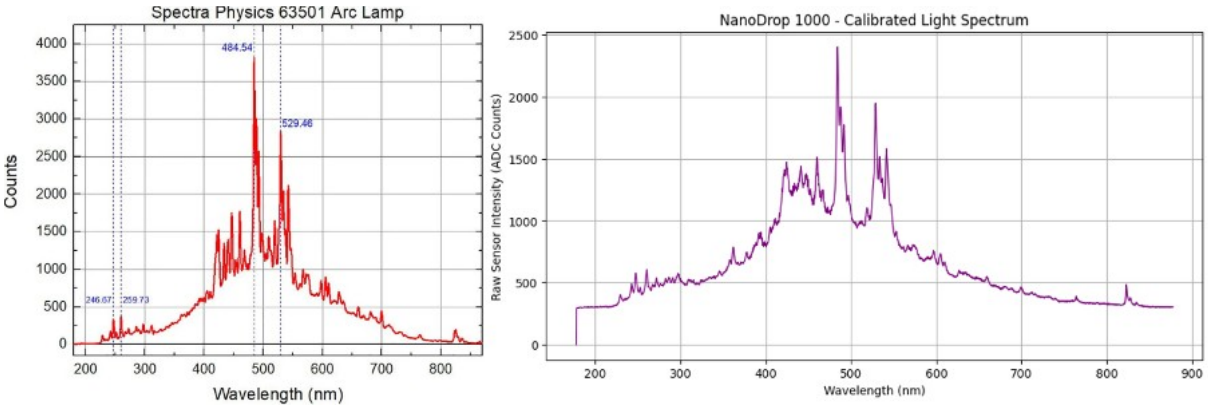


*Figure 3: A xenon flash lamp spectrum as uploaded by Esser et al. on the **left** compared to the xenon flash lamp spectrum measured by the custom script on the **right**.*

The measured spectrum is near identical to what is found in literature as shown by the characteristic peaks in the UV region. The central region also seems to be similar with the sharp peaks and the triplet looking structure at ~490 nm, as well as the larger signal at around 540 nm. Lastly the small bumps near the red and IR region also correspond very well, further proving the accuracy of the custom code.

As those first tests were successful, we moved into integrating this system into PyLabRobot. Once that was completed, we tested the instrument for the detection of paracetamol (acetaminophen) in water.

As we did before, we first measured a blank (Figure 4). Every absorbance measurement is divided against a blank. The green trace is the blank, light from the xenon lamp passes through transparent water, so it shows the lamp's own emission profile (brightest in the blue-green, ~484 nm), and not the spectrum of water. The grey trace is the dark frame (~292 counts), the detector's baseline with the lamp off. All absorption measurements are calculated as follows:

Equation 1: Absorbance from measured intensities

$$A = -\log_{10}\left(\frac{I_{sample} - I_{dark}}{I_{blank} - I_{dark}}\right)$$

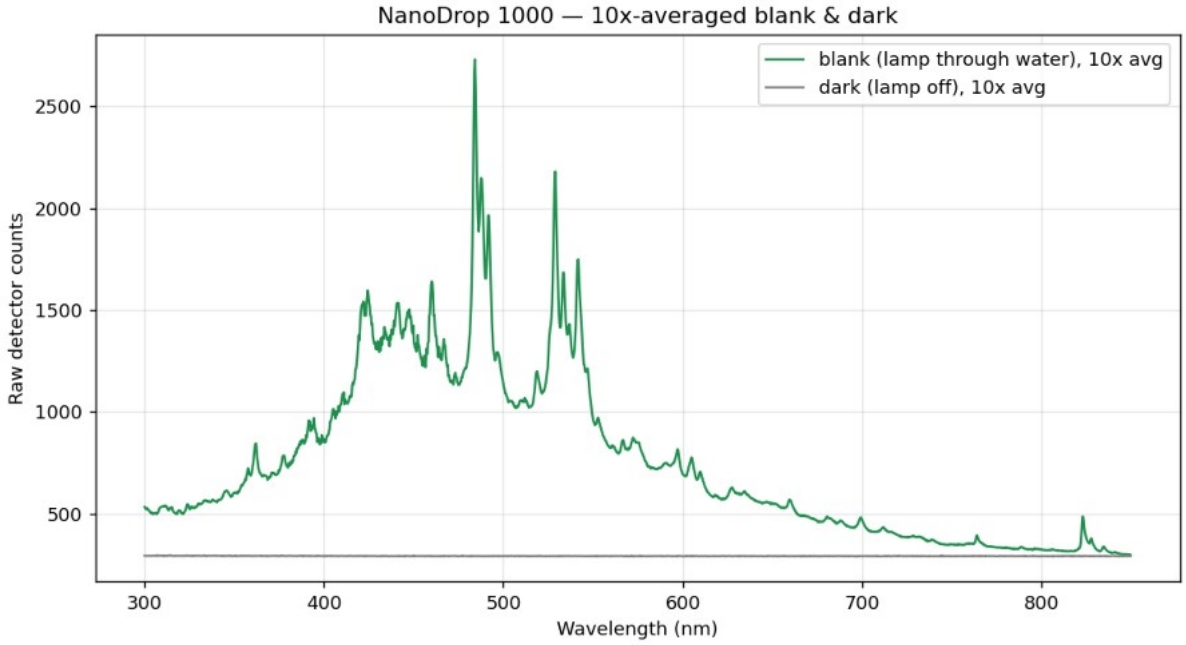


Figure 4: 10x Averaged blank measurement using water.

This blank is 10 scans averaged, acquired within a single pedestal-down / lamp-on cycle (the way the vendor software works). It is worth noting that a raw blank is already smooth, as its high signal (hundreds to ~2700 counts) means that relative noise is small. Therefore, averaging it barely changes how the blank curve looks. Thus, the payoff of averaging is not visible here; it shows up in the absorbance, where you divide small, noisy numbers.

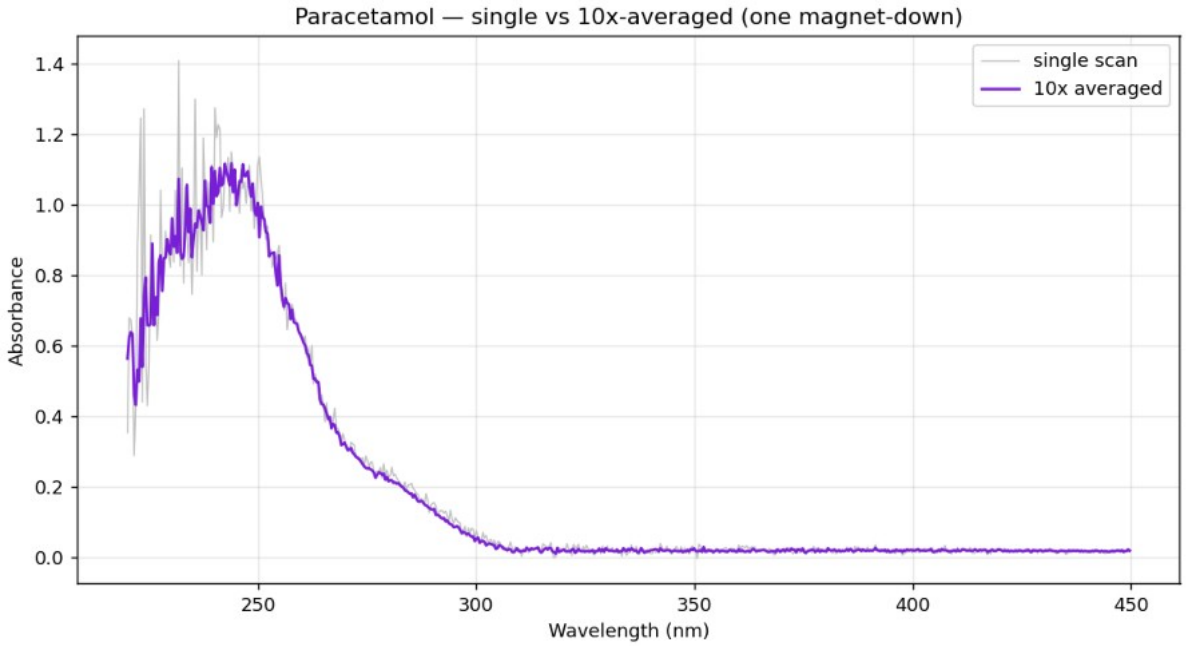


Figure 5: Paracetamol spectrum as measured by the ND1000 in PyLabRobot. 10x averaged.

Figure 5 shows the paracetamol absorbance spectrum, a single scan (grey) vs the 10× averaged (purple) are shown and here the averaging clearly matters. The well-known strong UV band (paracetamol's λmax ≈ 243 nm) is resolved; the visible region is flat because paracetamol is colourless. Averaging cleans the mid-band and baseline substantially. The residual jitter at the far-UV edge (< 235 nm) is a hardware limit, since the lamp emits almost no light there, which means those pixels are photon-starved, and no amount of averaging fully rescues them. It is worth noting that the remaining polynomial coefficients discussed earlier should apply smoothing, yet it remains to be seen if it can help in this scenario.

Figure 6 shows two smoothing options for the 10x averaged graph. The first, Boxcar, is standard in Ocean Optics software, it takes an average over adjacent pixels (the Boxcar width, here roughly 9 nm) and averages the results. The second is the Savitzky–Golay (SG) method. This method uses a local polynomial fit and appears to preserve the peak height position better than boxcar. It smooths over approximately 8 nm. For the broad band of paracetamol, both are safe to use, however SG seems to portray what is physically detected more accurately.

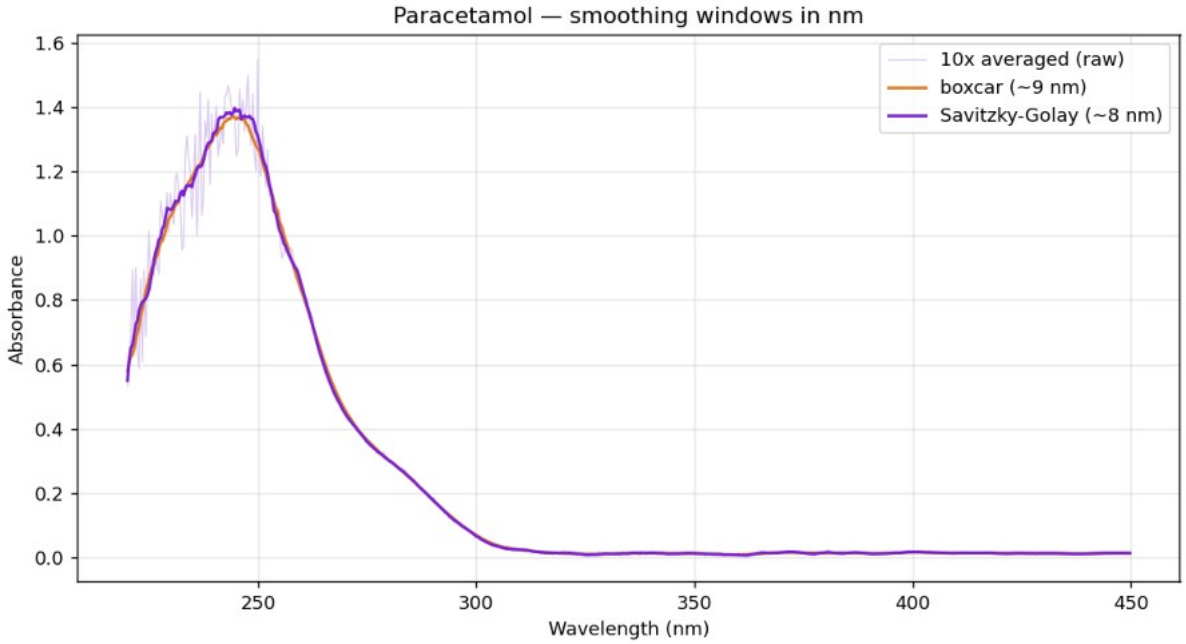


Figure 6: Two smoothing filters were applied to the 10×-averaged spectrum, windows are expressed in nm (the physically meaningful unit; the filter functions take a window in detector pixels, here converted via 0.368 nm/pixel)

Conclusions

In this project we set out to answer a simple question: can a legacy laboratory instrument, stranded by outdated proprietary software, be brought back to life? Using the Thermo Fisher NanoDrop 1000 as a pilot, we showed that the answer is a resounding yes.

Through black-box reverse engineering, we decoded the instrument's undocumented USB communication protocol. We identified the internal Ocean Optics USB2000 spectrometer and mapped the endpoints responsible for commands, status, and image data. We extracted the wavelength calibration coefficients from the EEPROM, reconstructed the 12-bit pixel data from interleaved byte blocks, and identified the commands for the xenon lamp and the proprietary solenoid clamp. On this basis we wrote our own open-source Python driver and integrated it into PyLabRobot, making the NanoDrop controllable from any modern machine and from within a broader automated laboratory.

The instrument was validated in two ways. First, the measured xenon lamp spectrum closely matched the reference spectrum from the literature, confirming that our decoding of the raw data is correct. Second, the instrument successfully detected paracetamol in water, with blank subtraction, signal averaging, and smoothing producing clean, interpretable absorbance spectra.

Beyond the technical results, this project demonstrates a broader principle: the hardware of "legacy" instruments is often still perfectly functional, it is only the software that is obsolete. By reverse engineering and open-sourcing the control software, we can extend the life of these instruments, reduce e-waste, and give laboratories full ownership of both their instruments and their data. By integrating the NanoDrop into the PyLabRobot ecosystem, we make this capability available to everyone, contributing to the vision of open, self-driving laboratories.

Future work may include auto-exposure, non-linearity correction using the 7th-order polynomial, and thermal drift compensation. Furthermore, validating the device on different substances will strengthen our repertoire and show the device is fully functional. Lastly protocols should be developed, which automatically do the correct set of actions to take the measurement for the desired outcome, e.g. protein or DNA analysis. **We hope this pilot encourages other groups to revive their own legacy instruments rather than discard them!**

Links to the project:

GitHub ND1000 pylabrobot demo https://github.com/vcjdeboer/nanodrop-pylabrobot-demo
GitHub PyLabRobot merge https://github.com/PyLabRobot/pylabrobot/pull/1166
GitHub OSE ND1000 https://github.com/Jordvl/Pylabrobot-Open-science-ND1000
Zenodo raw data: https://doi.org/10.5281/zenodo.22695032

Acknowledgement:

This work was supported by the WUR Open Science & Education programme (grant no. OSE202534).

Sources

1. Morales, J. Manufacturer issues remote kill command to disable smart vacuum after engineer blocks it from collecting data. Tom's Hardware (2025). https://www.tomshardware.com/tech-industry/big-tech/manufacturer-issues-remote-kill-command-to-nuke-smart-vacuum-after-engineer-blocks-it-from-collecting-data-user-revives-it-with-custom-hardware-and-python-scripts-to-run-offline
2. EV owner stranded abroad as remote lock leaves car's smart system disabled for over 30 hours. SHINE (Shanghai Daily) (2026). https://www.citynewsservice.cn/articles/shanghaidaily/news/ev-owner-stranded-abroad-as-remote-lock-leaves-cars-smart-system-disabled-for-over-30-hours-zmabyrjn
3. The TechBuzz Team. Cloud storage nightmare: user loses 30 years of data after Microsoft account lock. TechBuzz AI. https://www.techbuzz.ai/articles/cloud-storage-nightmare-user-loses-30-years-of-data-after-microsoft-account-lock
4. Still booting after all these years: the people stuck using ancient Windows computers. BBC Future (2025). https://www.bbc.com/future/article/20250516-the-people-stuck-using-ancient-windows-computers
5. Reverse engineering. Wikipedia (2026). https://en.wikipedia.org/wiki/Reverse_engineering
6. Scherrer, T. (OZ2CPU). NanoDrop spectrophotometer ND-1000 2006 test teardown (2024). https://www.youtube.com/watch?v=2F05fXeli7M
7. Beagle USB 12 protocol analyzer. Total Phase. https://www.totalphase.com/products/beagle-usb12/
8. Wireshark. Wireshark Foundation. https://www.wireshark.org/
9. I have successfully installed the ThermoFisher NanoDrop 1000 under Windows 10 64bit. ResearchGate. https://www.researchgate.net/post/I_have_successfully_installed_the_ThermoFisher_NanoDrop_1000_under_Windows_10_64bit
10. Ocean Optics. USB2000+ data sheet
11. Microsoft OS 2.0 descriptors specification. Microsoft Learn. https://learn.microsoft.com/en-us/windows-hardware/drivers/usbcon/microsoft-os-2-0-descriptors-specification
12. Jordvl. Pylabrobot-Open-science-ND1000 (source code). GitHub. https://github.com/Jordvl/Pylabrobot-Open-science-ND1000/tree/main/Code
13. Ocean Optics. USB2000 fiber optic spectrometer installation and operation manual. https://www.oceanoptics.com/wp-content/uploads/2024/12/USB2000Plus-Operating-Instructions.pdf